Name- Vivek Anand
Grp- KRG_2B
UID - 23BCS11598

(Q) Given three integers n, a and b. Return n^{th} magical number. Answer may be very large, return ans % 10^9+7

Magical no. - If no. either divisible by a or b.

n=1, a=2, b=3
output=2

→ Approach:

(Brute force)
The lowest can be min(a,b) when n=1
& highest can be n* min(a,b) when n>1

Edge case: If (n==1) return min(a,b).

low= min(a,b) ; high = n * min(a,b) ;
counter = 1;

for (i → low → high){
if (i % a == 0) counter ++;
else if (i % b == 0) counter ++;
if (counter == n) return i % 10^9+7
}

T.C. - O(n) [Takes more time than O(n) & overflows when product of $10^5 * 10^5$]
S.C. - O(1)

Code:

```
int magicalNum (int n, int a, int b){

  if (n==1) return min(a,b);

  int low= min(a,b), high=n* min(a,b);
  int counter=0;
  for(int i=low; i<high; i++){

    if (i% a==0) counter ++;
else if (i% b==0) counter ++;

    if (counter==n){
      return i%(1e9+7); i %(1e9+7);
    }
  }
  return -1;
}
```

For large numbers, int will overflow, long long should work but still due to linear search, it would take more time.

Since, the number are in sorted manner, binary search can be applied.

Approach 2:

low = min (a, b) ; high = n * min (a, b)

lcm = a*b / gcd (a, b);

mid → binary search

$$magical_count = \frac{mid}{a} + \frac{mid}{b} - \frac{mid}{lcm(a,b)}$$

if (magical_count) ≥ n
    high = mid
else if (magical_count) < n
    low = mid + 1

return low % 1e9 + 7

Code:

```
int magicalNum (int n, int a, int b){

  int low = min (a, b);
  int high = n* low;
  int lcm = a*b / gcd (a, b);
  //Assuming gcd code is already given

  int mid;

  while (low < high){
    mid = low + ((high - low)/2);

  int magical_count = (mid/a) + (mid/b) - (mid/lcm);
```

```
if (magical_count >= n) mid=high;
else if (magical_count < n) low=mid+1;
}

return low % (1e9+7);
}
```

T.C. - O(log(n*min(a, b)))
S.C. - O(1)